

CAHIER DES CHARGES

Gabarit étudiant - Projet intégrateur

Noms des étudiant(e)s	Boukik Abbes / Boukhdiche Abdellah / Hammouche Yacine
Nom du projet	Smartpharma
Programme	Programmation Informatique
Cours	Projet intégrateur (IFM30747 – 0010 – P2026)
Professeur.e	Stéphanie Ntumba Kahindo
Date de remise	01/08/2026

La Cité - Collège d'arts appliqués et de technologie

1. Présentation du projet

1.1 Contexte

Le secteur pharmaceutique exige une gestion rigoureuse des médicaments, des stocks, des ventes, des clients et des fournisseurs afin d'assurer un service efficace et sécuritaire. Dans plusieurs pharmacies, certaines opérations sont encore réalisées manuellement ou à l'aide de plusieurs outils distincts, ce qui augmente les risques d'erreurs, de perte de temps, de rupture de stock et de mauvaise gestion des données.

Le projet **SmartPharma** consiste à développer une application de bureau permettant de centraliser les principales opérations d'une pharmacie au sein d'une seule plateforme. L'application facilite la gestion des médicaments, le suivi des stocks, l'enregistrement des ventes, ainsi que la gestion des clients et des fournisseurs grâce à une interface conviviale et une base de données relationnelle.

SmartPharma est développé avec les technologies **C#**, **Windows Presentation Foundation (WPF)**, **Entity Framework Core** et **SQL Server**. Ces technologies permettent d'offrir une application performante, fiable et facile à maintenir.

Grâce à une interface moderne et intuitive, SmartPharma contribue à améliorer l'efficacité du personnel, à réduire les erreurs de gestion et à assurer un meilleur suivi des informations liées aux activités de la pharmacie.

Ce projet est réalisé dans le cadre du cours **Projet intégrateur** du programme **Programmation Informatique** au Collège La Cité. Il permet de mettre en pratique les connaissances acquises en programmation orientée objet, en conception d'interfaces graphiques, en bases de données relationnelles, en développement d'applications et en gestion de projet Agile.

1.2 Objectifs

1.2.1 Objectif général

Développer une application de bureau complète permettant d'améliorer la gestion quotidienne d'une pharmacie en automatisant les principales opérations liées aux médicaments, aux stocks, aux ventes, aux clients et aux fournisseurs, tout en offrant une interface simple, moderne et efficace.

1.2.2 Objectifs spécifiques

Le projet SmartPharma vise à atteindre les objectifs suivants :

- Développer une interface graphique conviviale et intuitive en utilisant **WPF**.
- Permettre la gestion complète des médicaments (ajout, modification, suppression et recherche).
- Assurer le suivi des stocks et permettre la mise à jour des quantités disponibles.
- Développer un module de gestion des ventes avec calcul automatique du montant total et mise à jour du stock.
- Permettre la gestion des clients et de leurs informations.
- Permettre la gestion des fournisseurs et de leurs informations.
- Enregistrer automatiquement les données dans une base de données **SQL Server** grâce à **Entity Framework Core**.
- Réduire les erreurs de gestion grâce à l'automatisation des opérations.
- Concevoir une application évolutive permettant l'ajout futur de nouvelles fonctionnalités, telles que l'authentification des utilisateurs, les rapports statistiques et la gestion des rôles.

1.3.1 Fonctionnalités incluses

Liste ce qui SERA développé dans ton projet. Sois précis - ce que tu mets ici, tu devras le livrer.

Fonctionnalité	Description courte
Authentification	Connexion sécurisée des utilisateurs avec rôles.
Gestion des médicaments	Ajouter, modifier, supprimer et consulter les médicaments.
Gestion des stocks	Mettre à jour automatiquement les quantités disponibles.
Gestion des ventes	Enregistrer chaque vente réalisée.
Recherche	Rechercher rapidement un médicament ou un fournisseur.
Gestion des utilisateurs	Créer et administrer les comptes utilisateurs.
Tableau de bord	Afficher les statistiques principales de la pharmacie.
Gestion des fournisseurs	Ajouter, modifier et consulter les fournisseurs.

1.3.2 Fonctionnalités exclues

Liste ce qui NE SERA PAS dans ton projet (fonctionnalités trop complexes, hors scope, remises à plus tard).

Fonctionnalité exclue	Raison
Païement en ligne	Hors du périmètre du MVP.
Application mobile	Mode utilisations au milieu de travail
Intelligence artificielle	Fonctionnalité trop complexe pour le projet académique.

1.3.3 Limites du MVP

La première version (**MVP – Minimum Viable Product**) de **SmartPharma** est conçue pour répondre aux besoins essentiels de la gestion quotidienne d'une pharmacie. Elle inclut les principales fonctionnalités développées durant le Sprint 2, notamment la gestion des médicaments, du stock, des ventes, des clients et des fournisseurs.

Afin de respecter les contraintes de temps et le périmètre du projet académique, certaines fonctionnalités avancées ne sont pas incluses dans cette première version. Parmi celles-ci figurent l'authentification des utilisateurs, la gestion des rôles (administrateur, pharmacien, assistant), les rapports et statistiques avancés, l'impression des factures, les alertes automatiques pour les médicaments expirés ou les stocks faibles, ainsi que la gestion de plusieurs pharmacies.

Malgré ces limites, le MVP constitue une application fonctionnelle permettant de démontrer les principales opérations de gestion d'une pharmacie. Son architecture basée sur **C#**, **WPF**, **Entity Framework Core** et **SQL Server** est conçue pour être évolutive, facilitant ainsi l'intégration de nouvelles fonctionnalités lors des prochains sprints, notamment le Sprint.

2. Description fonctionnelle

2.1 Besoins et public cible

2.1.1 Problématique

La gestion quotidienne d'une pharmacie nécessite le suivi de nombreuses informations, telles que les médicaments, les stocks, les ventes, les clients et les fournisseurs. Lorsque ces opérations sont réalisées manuellement ou à l'aide de plusieurs outils distincts, elles augmentent les risques d'erreurs de saisie, de perte de données, de rupture de stock et de difficultés à retrouver rapidement les informations nécessaires.

Ces problèmes peuvent réduire l'efficacité du personnel et nuire à la qualité du service offert aux clients. Par exemple, un mauvais suivi du stock peut entraîner une rupture de médicaments essentiels, tandis qu'une gestion inefficace des ventes peut compliquer le suivi des transactions et des revenus.

Le projet **SmartPharma** répond à cette problématique en proposant une application de bureau centralisée permettant de gérer les principales activités d'une pharmacie au sein d'une seule plateforme. Grâce à cette solution, les utilisateurs peuvent consulter rapidement les informations, enregistrer les ventes, gérer les médicaments, suivre le stock et administrer les clients ainsi que les fournisseurs de manière simple et efficace.

2.1.2 Public cible

L'application **SmartPharma** est destinée aux professionnels travaillant dans une pharmacie.

Pharmacien

Le pharmacien est le principal utilisateur de l'application. Il peut gérer les médicaments, consulter les quantités disponibles, effectuer les ventes, mettre à jour les informations des médicaments, gérer les clients et les fournisseurs.

Assistant en pharmacie

L'assistant peut consulter les médicaments disponibles, effectuer les ventes, rechercher un médicament, consulter le stock et enregistrer les informations des clients selon les besoins quotidiens de la pharmacie.

Gestionnaire de la pharmacie

Le gestionnaire utilise l'application pour superviser les médicaments, consulter les informations relatives au stock, gérer les fournisseurs et suivre les ventes réalisées afin d'assurer le bon fonctionnement de la pharmacie.

2.2 Fonctionnalités principales

2.2.1 Gestion des médicaments

Cette fonctionnalité permet de gérer l'ensemble des médicaments disponibles dans la pharmacie.

L'utilisateur peut :

- Ajouter un médicament.
- Modifier les informations d'un médicament.
- Supprimer un médicament.
- Rechercher un médicament.

- Consulter la liste complète des médicaments.
- Vérifier les quantités disponibles.

Chaque médicament contient les informations suivantes :

- Nom
 - Prix
 - Quantité en stock
 - Quantité par boîte
 - Forme pharmaceutique (comprimé, gélule, sirop, etc.)
 - Date d'expiration
-

2.2.2 Gestion du stock

Le module de gestion du stock permet d'assurer un suivi des quantités disponibles pour chaque médicament.

L'utilisateur peut :

- Consulter les quantités disponibles.
- Modifier la quantité en stock.
- Rechercher un médicament.
- Identifier rapidement les médicaments dont le stock est inférieur à 10 unités.

Cette fonctionnalité facilite le suivi des stocks et aide à anticiper les ruptures.

2.2.3 Gestion des ventes

Le module des ventes permet d'enregistrer les ventes réalisées dans la pharmacie.

Lors d'une vente :

- l'utilisateur sélectionne un médicament ;
- le prix du médicament est affiché automatiquement ;
- la date d'expiration est affichée ;
- la quantité disponible est indiquée ;
- l'utilisateur saisit la quantité vendue ;
- le montant total est calculé automatiquement.

Une fois la vente enregistrée :

- la transaction est sauvegardée dans la base de données ;
 - le stock du médicament est automatiquement mis à jour.
-

2.2.4 Gestion des clients

Cette fonctionnalité permet de gérer les informations des clients de la pharmacie.

L'utilisateur peut :

- Ajouter un client.
- Modifier les informations d'un client.
- Supprimer un client.
- Rechercher un client.
- Consulter la liste complète des clients.

Pour chaque client, les informations suivantes sont enregistrées :

- Nom
 - Téléphone
 - Adresse courriel
 - Adresse
-

2.2.5 Gestion des fournisseurs

Cette fonctionnalité permet de gérer les fournisseurs partenaires de la pharmacie.

L'utilisateur peut :

- Ajouter un fournisseur.
- Modifier les informations d'un fournisseur.
- Supprimer un fournisseur.
- Rechercher un fournisseur.
- Consulter la liste complète des fournisseurs.

Pour chaque fournisseur, les informations suivantes sont conservées :

- Nom
- Nom de l'entreprise
- Téléphone
- Adresse courriel
- Adresse

2.2.6 Tableau de bord

Le tableau de bord constitue la page principale de l'application.

Il permet à l'utilisateur d'accéder rapidement aux différents modules :

- Gestion des médicaments
- Gestion du stock
- Gestion des ventes
- Gestion des clients
- Gestion des fournisseurs
- Contact

Cette organisation facilite la navigation et améliore l'expérience utilisateur.

2.3 Interface utilisateur

2.3.1 Structure générale

L'interface utilisateur de SmartPharma est organisée autour d'un tableau de bord moderne permettant d'accéder rapidement aux principales fonctionnalités.

Le menu principal comprend les modules suivants :

- Médicaments
- Stock
- Ventes
- Clients
- Fournisseurs
- Contact

Chaque module possède sa propre fenêtre avec une interface uniforme comprenant :

- un titre ;
- une barre de recherche ;
- des boutons d'action (Ajouter, Modifier, Supprimer, Rechercher) ;
- un tableau d'affichage des données ;
- un bouton Retour vers le tableau de bord.

Les formulaires d'ajout et de modification utilisent un **ScrollView** afin de garantir une bonne accessibilité lorsque le contenu dépasse la taille de la fenêtre.

2.3.2 Technologies d'interface

L'interface utilisateur est développée avec les technologies suivantes :

- **Windows Presentation Foundation (WPF)** pour la création des interfaces graphiques.
- **XAML** pour la conception des fenêtres et des contrôles.
- **C#** pour la logique de l'application.
- **Entity Framework Core** pour la communication avec la base de données SQL Server.

Ces technologies permettent de créer une application moderne, fluide et facile à maintenir.

2.3.3 Maquettes et prototypes

Les maquettes réalisées représentent les principales interfaces de l'application, notamment :

- Tableau de bord principal.
- Gestion des médicaments.
- Gestion du stock.
- Gestion des ventes.
- Gestion des clients.
- Gestion des fournisseurs.
- Fenêtres d'ajout et de modification des données.

Les captures d'écran de ces interfaces seront présentées en annexe du document.

2.4 Conditions d'utilisation

Pour utiliser SmartPharma, les conditions suivantes doivent être respectées :

- Disposer d'un ordinateur fonctionnant sous Windows.
- Installer l'application SmartPharma.
- Avoir accès à une instance de **SQL Server LocalDB** ou **SQL Server** contenant la base de données de l'application.
- Utiliser une version récente de l'application.
- Disposer des droits nécessaires pour accéder à la base de données.

2.5 Exigences fonctionnelles

#	Exigence fonctionnelle
EF-001	Le système doit permettre la connexion des utilisateurs.
EF-002	Le système doit permettre d'ajouter un médicament.
EF-003	Le système doit permettre de modifier un médicament.
EF-004	Le système doit permettre de supprimer un médicament.
EF-005	Le système doit permettre de rechercher un médicament.
EF-006	Le système doit gérer automatiquement le stock.
EF-007	Le système doit enregistrer les ventes.
EF-008	Le système doit gérer les fournisseurs.
EF-009	Le système doit gérer les utilisateurs.
EF-010	Le système doit afficher un tableau de bord.

2.6 Exigences non fonctionnelles

Ce sont les qualités du système : performance, sécurité, accessibilité, disponibilité, compatibilité. Utilise le format ENF-XXX.

#	Catégorie	Exigence
ENF-001	Performance	Les pages doivent s'afficher en moins de 2 secondes.
ENF-002	Sécurité	Les utilisateurs doivent être authentifiés avant l'accès au système.
ENF-003	Fiabilité	Les données doivent être enregistrées sans perte.
ENF-004	Disponibilité	L'application doit être disponible pendant les heures d'ouverture.
ENF-005	Ergonomie	L'interface doit être simple et intuitive.
ENF-006	Compatibilité	L'application doit fonctionner sous Windows 10 et Windows 11.

3. Description technique

3.1 Technologies à utiliser

Le développement de SmartPharma repose sur plusieurs langages de programmation, chacun ayant un rôle précis dans le fonctionnement de l'application.

3.1.1 Langages de programmation

Quels langages utiliseras-tu et pourquoi ?

Langage	Utilisation
C#	Développement de la logique métier et des fonctionnalités.
XAML	Création de l'interface graphique WPF.
SQL	Création et manipulation des tables Cassandra.

3.1.2 Frameworks et bibliothèques

Liste les frameworks et bibliothèques pour le frontend et le backend.

Technologie	Rôle	Frontend / Backend
WPF	Interface graphique	Frontend
.NET	Développement de l'application	Backend
Material Design in XAML	Interface moderne	Frontend
DataSBase SQL	Communication avec SQL	Backend

Base de données

Le projet **SmartPharma** utilise **SQL Server** comme système de gestion de base de données relationnelle. Les données sont stockées de manière sécurisée et sont accessibles à l'application grâce à **Entity Framework Core**, qui assure la communication entre le code C# et la base de données.

SQL Server offre une solution fiable et performante pour la gestion des informations de la pharmacie. Il permet de stocker un grand volume de données tout en garantissant leur intégrité, leur cohérence et un accès rapide lors des opérations d'ajout, de modification, de suppression et de consultation.

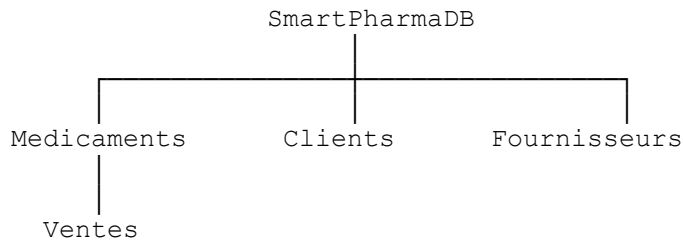
Les principales tables de la base de données sont les suivantes :

	Description
Medicaments	Contient les informations relatives aux médicaments (nom, prix, quantité en stock, quantité par boîte, forme pharmaceutique et date d'expiration).
Clients	Enregistre les informations des clients de la pharmacie (nom, téléphone, courriel et adresse).

Description

Fournisseurs	Contient les informations des fournisseurs partenaires (nom, entreprise, téléphone, courriel et adresse).
Ventes	Enregistre les ventes effectuées, incluant le médicament vendu, la quantité, la date de vente et le montant total.

Le diagramme ci-dessous illustre l'organisation générale de la base de données :



Chaque table est reliée à l'application par le **SmartPharmaDbContext**, qui permet d'effectuer les opérations **CRUD (Créer, Lire, Modifier et Supprimer)** à l'aide d'**Entity Framework Core**. Toutes les modifications réalisées dans l'application sont automatiquement enregistrées dans **SQL Server**, garantissant ainsi la cohérence et la persistance des données.

3.1.4 Outils de développement

Ex. : Git, GitHub, VS Code, Jira, ...

Outil	Utilisation
Visual Studio	Développement de l'application
GitHub	Hébergement du projet
Jira	Gestion des tâches et des sprints
SQL Server	Hébergement de la base Sql server

3.2 Architecture du système

3.2.1 Type d'architecture

SmartPharma est développé selon une architecture en couches (architecture client-serveur pour une application de bureau), permettant de séparer l'interface utilisateur, la logique métier et l'accès aux données. Cette organisation facilite le développement, la maintenance et l'évolution de l'application.

L'architecture repose sur les composants suivants :

- **Interface utilisateur (WPF)** : permet aux utilisateurs d'interagir avec l'application.
- **Logique métier (C#)** : traite les opérations demandées par l'utilisateur.

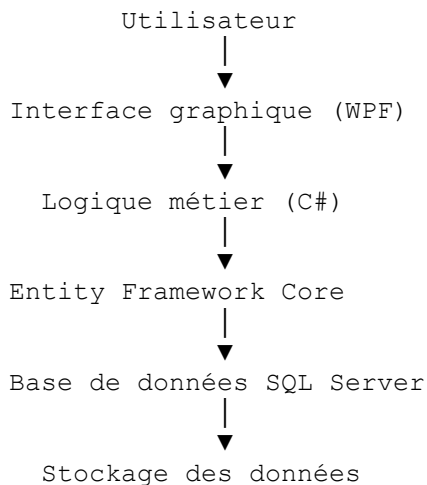
- **Entity Framework Core** : assure la communication entre l'application et la base de données.
- **SQL Server** : stocke les informations de manière permanente.

Cette architecture présente plusieurs avantages :

- séparation claire des responsabilités ;
- facilité de maintenance ;
- meilleure évolutivité ;
- accès simplifié à la base de données ;
- amélioration de la fiabilité de l'application.

3.2.2 Organisation générale

L'architecture générale de SmartPharma est représentée par le schéma suivant :



Fonctionnement du système

Le fonctionnement général de l'application est le suivant :

1. L'utilisateur effectue une action à partir de l'interface graphique (ajout, modification, suppression, recherche ou vente).
2. La demande est transmise à la logique métier développée en C#.
3. La logique métier effectue les validations nécessaires et prépare les données.
4. Entity Framework Core traduit automatiquement les opérations en requêtes SQL.
5. SQL Server enregistre ou récupère les informations demandées.

6. Les résultats sont renvoyés à l'application et affichés à l'utilisateur.

Cette architecture permet une communication efficace entre les différentes couches du système tout en assurant une bonne organisation du code.

3.2.3 Sécurité

La première version de SmartPharma met en place plusieurs mécanismes afin d'assurer la fiabilité des données et de limiter les erreurs de manipulation.

Validation des données

Avant chaque enregistrement, les informations saisies sont validées afin de vérifier qu'elles sont complètes et cohérentes. Par exemple :

- les champs obligatoires doivent être remplis ;
- les prix doivent être positifs ;
- les quantités doivent être valides ;
- une date d'expiration doit être sélectionnée lors de l'ajout d'un médicament.

Ces validations permettent de réduire les erreurs de saisie et d'assurer la qualité des données enregistrées.

Protection des données

Toutes les informations de l'application sont enregistrées dans une base de données **SQL Server**, ce qui garantit leur persistance et leur intégrité. Les opérations d'ajout, de modification et de suppression sont réalisées par **Entity Framework Core**, ce qui réduit les risques d'erreurs lors de l'accès aux données.

Sauvegarde des données

Les données sont stockées dans une base de données relationnelle SQL Server. Elles peuvent être sauvegardées régulièrement à l'aide des outils de gestion de SQL Server afin de prévenir toute perte d'information.

Évolutions prévues

Les fonctionnalités de sécurité suivantes ne sont pas encore implémentées dans cette version et seront développées lors du **Sprint 3** :

- authentification des utilisateurs ;
- gestion des rôles (administrateur, pharmacien et assistant) ;
- journalisation des opérations importantes ;
- contrôle des accès selon les permissions des utilisateurs.

4. Planification et livrables

4.1 Phases du projet

Décris tes sprints ou phases de travail avec les dates et les activités prévues pour chacun.

Phase / Sprint	Période	Activités principales
Sprint 1	18 mai 2026 14 juin 2026	Définition du projet SmartPharma, analyse des besoins, rédaction du cahier des charges, création du projet Jira et du dépôt GitHub, planification (diagramme de Gantt), conception fonctionnelle, réalisation des premières maquettes de l'interface et mise en place de la structure du projet WPF.
Sprint 2	15 Juin 2026 12 juillet 2026	Conception technique de l'application, création de la base de données SQL Server, configuration d'Entity Framework Core, développement des modules Gestion des médicaments, Gestion du stock, Gestion des ventes, Gestion des clients et Gestion des fournisseurs , amélioration de l'interface utilisateur, intégration des fonctionnalités CRUD, réalisation des premiers tests fonctionnels et correction des bogues identifiés.
Sprint 3	12 Juillet 2026 9 août 2026	Finalisation des fonctionnalités restantes, développement du module Rapports et statistiques , ajout de l'authentification et de la gestion des rôles (si le temps le permet), optimisation de l'application, réalisation des tests complets, correction des anomalies, rédaction de la documentation technique, préparation du manuel utilisateur, création de la vidéo de démonstration, préparation de la présentation finale et livraison du projet.

4.2 Livrables attendus

Livrable	Description	Date prévue
Documentation utilisateur	Document final	11 août
Présentation finale	Présentation PowerPoint du projet.	11 août
Code source	Projet complet disponible sur GitHub.	11 août

5. Modalités de validation

Afin de garantir la qualité, la fiabilité et le bon fonctionnement de **SmartPharma**, plusieurs types de tests seront réalisés tout au long du développement. Ces tests permettront de vérifier que les fonctionnalités développées répondent aux besoins des utilisateurs, que les données sont correctement enregistrées dans la base de données SQL Server et que l'application demeure stable et performante.

5.1 Tests fonctionnels

Les tests fonctionnels permettent de vérifier que chaque fonctionnalité développée fonctionne conformément aux exigences du projet.

Test 1 : Ajouter un médicament

Objectif

Vérifier que l'ajout d'un médicament fonctionne correctement.

Étapes

- Ouvrir le module **Gestion des médicaments**.
- Cliquer sur **Ajouter**.
- Remplir les informations du médicament.
- Cliquer sur **Enregistrer**.

Résultat attendu

Le médicament est ajouté à la liste et enregistré dans la base de données SQL Server.

Test 2 : Modifier un médicament

Objectif

Vérifier que la modification des informations d'un médicament fonctionne correctement.

Étapes

- Sélectionner un médicament.
- Cliquer sur **Modifier**.
- Modifier les informations souhaitées.
- Cliquer sur **Enregistrer**.

Résultat attendu

Les nouvelles informations sont enregistrées dans la base de données et immédiatement affichées dans la liste.

Test 3 : Supprimer un médicament

Objectif

Vérifier que la suppression d'un médicament fonctionne correctement.

Étapes

- Sélectionner un médicament.
- Cliquer sur **Supprimer**.
- Confirmer la suppression.

Résultat attendu

Le médicament est supprimé de la base de données SQL Server et disparaît de la liste.

Test 4 : Effectuer une vente

Objectif

Vérifier que l'enregistrement d'une vente fonctionne correctement.

Étapes

- Ouvrir le module **Gestion des ventes**.
- Sélectionner un médicament.
- Vérifier l'affichage automatique du prix.
- Saisir la quantité vendue.
- Cliquer sur **Enregistrer**.

Résultat attendu

- La vente est enregistrée.
 - Le montant total est calculé automatiquement.
 - Le stock du médicament est mis à jour automatiquement.
-

Test 5 : Gestion des clients

Objectif

Vérifier le bon fonctionnement du module Clients.

Étapes

- Ajouter un client.
- Modifier ses informations.
- Effectuer une recherche.
- Supprimer le client.

Résultat attendu

Toutes les opérations sont correctement enregistrées dans SQL Server.

Test 6 : Gestion des fournisseurs

Objectif

Vérifier le bon fonctionnement du module Fournisseurs.

Étapes

- Ajouter un fournisseur.
- Modifier ses informations.
- Effectuer une recherche.
- Supprimer le fournisseur.

Résultat attendu

Toutes les opérations sont correctement enregistrées dans SQL Server.

5.2 Tests techniques

Les tests techniques permettent de vérifier la qualité technique de SmartPharma ainsi que la communication entre les différentes composantes du système.

Vérification de la connexion à la base de données

Le système doit établir correctement la connexion avec **SQL Server** dès le démarrage de l'application.

Résultat attendu :

La connexion est établie sans erreur et les données sont accessibles.

Vérification des opérations CRUD

Les opérations suivantes doivent fonctionner correctement :

- Création
- Consultation
- Modification
- Suppression

pour les modules suivants :

- Médicaments
- Clients
- Fournisseurs

Les données doivent être immédiatement enregistrées dans **SQL Server** grâce à **Entity Framework Core**.

Vérification du module Ventes

Le système doit :

- afficher automatiquement le prix du médicament sélectionné ;
 - afficher la date d'expiration ;
 - afficher le stock disponible ;
 - calculer automatiquement le montant total ;
 - mettre à jour automatiquement le stock après chaque vente.
-

Gestion des erreurs

Le système doit afficher un message clair lorsqu'une erreur est détectée.

Exemples :

- champ obligatoire vide ;
 - quantité invalide ;
 - médicament introuvable ;
 - erreur de connexion à la base de données.
-

Validation des données

Avant chaque enregistrement, le système vérifie notamment :

- les champs obligatoires ;
- les valeurs numériques ;
- les prix positifs ;
- les quantités valides ;
- la sélection d'une date d'expiration ;

- la sélection de la forme du médicament.

5.3 Tests de performance

Les tests de performance permettent d'évaluer la rapidité et la stabilité de SmartPharma.

Élément testé	Critère attendu
Démarrage de l'application	moins de 3 secondes
Chargement de la liste des médicaments	moins de 2 secondes
Recherche d'un médicament	moins de 1 seconde
Ajout d'un médicament	moins de 2 secondes
Modification d'un médicament	moins de 2 secondes
Suppression d'un médicament	moins de 2 secondes
Enregistrement d'une vente	moins de 2 secondes
Chargement des clients	moins de 2 secondes
Chargement des fournisseurs	moins de 2 secondes

L'application devra conserver une bonne fluidité même avec plusieurs centaines de médicaments enregistrés dans la base de données.

5.4 Critères d'acceptation

Le projet **SmartPharma** sera considéré comme conforme lorsque les conditions suivantes seront satisfaites :

- Toutes les fonctionnalités prévues pour le Sprint 2 sont développées.
- Les médicaments peuvent être ajoutés, modifiés, supprimés et recherchés.
- Les clients peuvent être ajoutés, modifiés, supprimés et recherchés.
- Les fournisseurs peuvent être ajoutés, modifiés, supprimés et recherchés.
- Les ventes sont enregistrées correctement.
- Le montant total d'une vente est calculé automatiquement.
- Le stock est mis à jour automatiquement après chaque vente.
- Les données sont correctement enregistrées dans **SQL Server** via **Entity Framework Core**.
- Les principaux tests fonctionnels et techniques sont réussis.
- L'application fonctionne sans erreur critique.
- L'interface utilisateur est claire, intuitive et facile à utiliser.
- Le projet est documenté et respecte les exigences du cours.

6. Risques et contraintes

6.1 Risques identifiés

Identifie 4 à 6 risques qui pourraient nuire à ton projet, et explique comment tu prévois les gérer.

Risque	Impact	Probabilité	Plan de mitigation
Retard dans le développement des fonctionnalités	Élevé	Moyen	Planifier les tâches dans Jira, répartir équitablement le travail entre les membres de l'équipe et effectuer un suivi régulier de l'avancement à chaque sprint.
Présence de bogues dans l'application	Élevé	Moyen	Réaliser des tests fonctionnels après le développement de chaque fonctionnalité et corriger les anomalies avant de passer au module suivant.
Difficultés de communication au sein de l'équipe	Moyen	Faible	Organiser des réunions hebdomadaires, assurer un suivi des tâches dans Jira et maintenir une communication régulière entre les membres de l'équipe.
Perte ou corruption des données du projet	Élevé	Faible	Effectuer des sauvegardes régulières du projet sur GitHub et conserver une copie de la base de données SQL Server.

6.2 Contraintes du projet

Quelles contraintes s'appliquent à ton projet ? (temps, budget, ressources, technologies imposées, contexte académique...)

Contrainte	Description
Temps	Le projet doit être réalisé et livré avant la fin de la session, conformément au calendrier des sprints établi dans Jira.
Ressources	Le développement est réalisé à l'aide des outils et des ressources fournis dans le cadre du cours.
Technologies	Le projet doit être développé en utilisant les technologies imposées, soit C#, Windows Presentation Foundation (WPF), Entity Framework Core, SQL Server, GitHub et Jira.
Budget	Aucun budget n'est prévu pour ce projet. Toutes les technologies, logiciels et plateformes utilisés sont gratuits ou accessibles grâce aux licences académiques fournies par le Collège La Cité.
Travail d'équipe	La réussite du projet repose sur une bonne coordination entre les membres de l'équipe, une répartition équilibrée des tâches et une communication régulière afin de respecter les objectifs de chaque sprint.

7. Conclusion

Le projet **SmartPharma** a pour objectif de développer une application de bureau moderne permettant de faciliter la gestion quotidienne d'une pharmacie. Grâce aux fonctionnalités développées durant le projet, l'application permet de gérer efficacement les médicaments, le stock, les ventes, les clients et les fournisseurs au sein d'une interface simple, intuitive et conviviale.

Le développement de SmartPharma nous a permis de mettre en pratique les connaissances acquises durant le programme de **Programmation Informatique**, notamment en programmation orientée objet avec **C#**, en conception d'interfaces graphiques avec **Windows Presentation Foundation (WPF)**, en gestion de bases de données avec **SQL Server** et **Entity Framework Core**, ainsi qu'en gestion de projet à l'aide de **GitHub** et **Jira**.

Au terme du Sprint 2, les principales fonctionnalités de l'application sont opérationnelles. Les prochaines étapes consisteront à développer les fonctionnalités avancées prévues pour le Sprint 3, notamment les rapports et statistiques, l'authentification des utilisateurs, la gestion des rôles ainsi que l'optimisation générale de l'application.

Ce projet constitue une expérience enrichissante qui nous a permis de développer nos compétences techniques, organisationnelles et collaboratives tout en réalisant une application répondant à un besoin concret du domaine pharmaceutique.

8. Références

8.1 Documentation officielle

1. Microsoft. (2026). *Documentation C#*. <https://learn.microsoft.com/dotnet/csharp/>
2. Microsoft. (2026). *Documentation Windows Presentation Foundation (WPF)*. <https://learn.microsoft.com/dotnet/desktop/wpf/>
3. Microsoft. (2026). *Documentation Entity Framework Core*. <https://learn.microsoft.com/ef/core/>
4. Microsoft. (2026). *Documentation SQL Server*. <https://learn.microsoft.com/sql/>
5. Microsoft. (2026). *Documentation SQL Server Management Studio (SSMS)*. <https://learn.microsoft.com/sql/ssms/>
6. GitHub. (2026). *GitHub Docs*. <https://docs.github.com/>
7. Atlassian. (2026). *Jira Software Documentation*. <https://support.atlassian.com/jira-software-cloud/>
8. Microsoft. (2026). *Visual Studio Documentation*. <https://learn.microsoft.com/visualstudio/>
9. OpenAI. (2026). *ChatGPT* (utilisé pour l'aide à la rédaction, la reformulation et l'assistance au développement).

8.2 Notes de cours

- La Cité. (2026). *Cours de Développement d'applications de bureau (C#, WPF et XAML)*.
 - La Cité. (2026). *Cours de Base de données avancées (SQL Server et Entity Framework Core)*.
 - La Cité. (2026). *Cours de Projet intégrateur*.
-

8.3 Outils utilisés

- Visual Studio 2022
- C#
- Windows Presentation Foundation (WPF)
- Entity Framework Core
- SQL Server
- SQL Server Management Studio (SSMS)
- GitHub
- Jira Software
- diagrams.net (Draw.io)
- Microsoft Word
- Microsoft PowerPoint